Review of the *Specification-Driven Verification* Whitepaper

Overall Assessment

The implementation described by this paper appears to be substantially more mature than the manuscript itself communicates. This is not a proposal for a future system: the Specsaver implementation already exists, includes a working contract language, a testing framework, database theories, lowering to Rocq, and proved examples. Consequently, the primary weakness of the paper is not technical completeness but research presentation.

The manuscript currently reads like a detailed engineering whitepaper. To be competitive as a research publication (including venues such as ART), it should instead present the implementation as evidence supporting a set of precise scientific contributions.

The remainder of this review assumes that the implementation remains largely unchanged and focuses exclusively on improving the paper.

High-Level Recommendation

The paper should be rewritten around the central research idea rather than the implementation.

Currently the implied thesis is

> "Specsaver is a verification framework."

I believe the stronger thesis is

> "Executable behavioural contracts form a semantic intermediate language from which testing, runtime checking, differential validation, and machine-checked proof obligations can all be derived."

Specsaver then becomes the evidence that this thesis is practical.

This distinction significantly strengthens the novelty.

Primary Strengths

1. Clear Architectural Vision

The overall architecture is coherent.

Feature → Contract → Runtime Verification → Differential Testing → Proof Obligations

is easy to understand and represents a compelling unified workflow.

The paper avoids the common mistake of presenting verification as something disconnected from ordinary software engineering.

---

2. Real Implementation

The work is grounded in an actual implementation rather than an idealised design.

This dramatically increases credibility.

Rather than merely proposing

- contracts,
- database theories,
- semantic frames,
- lowering,

the system actually performs these operations.

The paper should make this much more explicit.

---

3. Excellent Choice of Examples

Using

- SQLite
- SQLAlchemy
- Logging
- OpenTelemetry

immediately differentiates the work from many verification papers that only reason about toy languages.

Similarly, the examples

- inventory
- transfers
- invitations

demonstrate increasing complexity in a convincing progression.

---

4. Semantic Frame Conditions

The frame system is probably one of the strongest contributions.

Rather than requiring explicit "everything else is unchanged" clauses, write paths generate the complement automatically.

This deserves much more emphasis.

---

5. Library Theories

The notion of decorating existing libraries with operational semantics and validating them differentially is novel enough to warrant significant discussion.

This should become one of the headline contributions.

Major Weaknesses

1. The Paper Reads Like Documentation

The dominant issue is stylistic.

Many sections describe

> what the system does

rather than explaining

> why this mechanism is interesting or scientifically significant.

A research paper should continually answer

Why is this difficult?

Why is this novel?

Why was this design chosen?

What alternatives exist?

---

2. Insufficient Formal Semantics

Many important concepts are introduced operationally.

For example

- semantic frames
- projection
- theory
- ghost state
- lowering

are described informally.

These deserve mathematical definitions.

For example

Definition (Projection)

Definition (Semantic Frame)

Definition (Theory)

Definition (Contract Satisfaction)

Definition (Ghost Transition)

The paper should present a semantic model before discussing implementation.

---

3. The Lowering Pipeline Is Underdeveloped

One of the most interesting aspects of the system is barely explained.

The reader currently observes

Python lambdas

↓

shape extraction

↓

generated Rocq

↓

proved obligations

This skips the most scientifically interesting component.

The paper should explain

Python AST

↓

normalisation

↓

contract calculus

↓

intermediate representation

↓

weakest precondition generation

↓

FunSpecS

↓

generated proof obligations

This is likely one of the primary technical contributions.

---

4. No Quantitative Evaluation

The implementation exists.

The paper should measure it.

For example

Metric	Value
Python LOC	
Rocq LOC	
Contracts	
Domains	
Generated obligations	
Generated theorems	
Manual proof lines	
Runtime verification time	
Lowering time	
Rocq compilation time	

Similarly,

measure fault detection by introducing mutations.

---

5. Too Few Explicit Scientific Claims

The paper contains many implicit contributions.

They should instead be stated directly.

For example

Contribution 1

Behaviour-first verification.

Contribution 2

Semantic frame inference.

Contribution 3

Differentially validated library theories.

Contribution 4

Symmetric projection architecture.

Contribution 5

Automatic lowering from executable contracts to Rocq proof obligations.

Readers should not have to infer novelty.

Trusted Computing Base

The paper should explicitly state what is trusted.

For example

Trusted

- Rocq kernel
- FunSpecS kernel
- weakest precondition calculus
- lowering implementation
- contract parser

Not yet proved

- correspondence between arbitrary Python implementations and Snakelet
- correctness of lowering
- soundness of differential testing beyond tested fragments

Reviewers appreciate clearly defined trust boundaries.

Differentiate Evidence from Future Work

Some sections describe implemented mechanisms.

Others describe research directions.

These should be separated more clearly.

Implemented

- executable contracts
- semantic frames
- runtime verification
- SQL theory
- logging theory
- OpenTelemetry theory
- lowering
- generated Rocq obligations

Research

- user-defined ghost algebras
- abstraction validation
- richer trace reasoning
- expanded supported language fragment

The reader should never wonder which components already exist.

Ghost Algebras

This section is interesting but currently overclaims.

The statement

> testing establishes the soundness of the abstraction

is too strong.

Testing provides empirical evidence, not proof.

Instead the paper should distinguish

Validation

The abstraction agrees with observed executions.

Soundness

The abstraction preserves semantics for all executions.

The former can be demonstrated experimentally.

The latter requires proof.

Suggested Additional Section

End-to-End Walkthrough

Include one complete example.

For example

Feature

↓

Scenario row

↓

Materialised SQLite database

↓

Contract

↓

Execution

↓

Projection

↓

Frame checking

↓

Generated intermediate contract

↓

Generated Rocq

↓

Machine-checked proof

This single narrative would substantially improve readability.

Evaluation Recommendations

Perform mutation testing.

Introduce faults such as

- omitted database update
- wrong account credited
- frame violation
- duplicate telemetry
- undeclared mutation
- unsupported SQL
- incorrect derived value

Then report which stage detects each error.

This demonstrates why combining runtime contracts, semantic frames, differential theories and proof obligations provides stronger assurance than any single mechanism.

Related Work

The comparison section should be expanded significantly.

In particular compare against

- Eiffel
- Dafny
- Nagini
- Frama-C
- Why3
- KeY
- TLA+
- Iris
- Property-based testing
- Behaviour-driven development

A useful structure is

Problem addressed

Approach

Limitations

How Specsaver differs

Suggested Theorems

The paper would benefit from formally stating properties even if some proofs are future work.

Examples

Frame Soundness

Everything outside the declared write frame remains observationally unchanged.

Projection Symmetry

The same projection function is sufficient for pre-state and post-state reasoning.

Theory Conformance

Differentially validated theories preserve observable behaviour over the supported fragment.

Lowering Correctness

The generated FunSpecS specification faithfully represents the accepted contract fragment.

Generator Totality

Every accepted contract either lowers successfully or is rejected loudly.

These clarify precisely what the system guarantees.

Suggested Reorganisation

1. Motivation

2. Scientific Contributions
3. Formal Model
4. Contract Language
5. Semantic Frames
6. Symmetric Projection
7. Theory Framework
8. Lowering Architecture
9. Evaluation
10. Related Work
11. Future Directions

The current ordering mixes engineering and semantics.

Overall Verdict

The implementation appears significantly stronger than the paper currently communicates.

The main improvements required are not additional features but stronger exposition.

Specifically the paper should

- emphasise the semantic contribution over the engineering;
- introduce formal definitions for the core concepts;
- explain the lowering pipeline in depth;
- present quantitative evaluation;
- define the trusted computing base;
- distinguish implemented components from research directions;
- strengthen related work;
- present explicit scientific claims rather than allowing readers to infer them.

With these revisions the work would read less like documentation for a sophisticated verification tool and more like a paper introducing a new methodology for specification-driven verification.